Python Tip: `any()` and `all()`

Need to know if **at least one** condition is true?

Want to check if **every** condition is true?

Use `any()` and `all()` for clean, readable checks!

Using `any()`

`any()` returns `True` if **at least one** element is true.

```
checks = [False, True, False]

# Don't do this
found = False
for check in checks:
    if check:
        found = True
print(found)

# Do this instead
print(any(checks))
```

```
True
True
```

Using `all()`

`all()` returns `True` only if **every** item is true.

```
checks = [True, True, True]

print(all(checks))

checks = [True, False, True]

print(all(checks))
```

```
True
False
```

Checking Conditions

Wrap them around a condition to see if **any** or **all** elements meet it.

```
scores = [82, 95, 67, 88]

# Did anyone score 90 or higher?
print(any(score >= 90 for score in scores))

# Did everyone pass?
print(all(score >= 60 for score in scores))
```

```
True
True
```

Empty Collections

With no items, `any()` is `False`, while `all()` is `True`.

```
items = []

print(any(items))
print(all(items))

# There is no true item for any() to find.
# There is no false item for all() to find.
```

```
False
True
```

Wrap-Up

Use `any()` when **at least one** condition must be true and `all()` when **every** condition must be true!

Follow me for more tips.

Shep Bryan IV